Dojo Manager — Technical & Testing Document

> **How to use this file:** This is your Technical & Testing Document draft, written to match the PAT rubric section 4 (15 marks). Copy it into Word/Google Docs, build a title page + table of contents, and **replace the *[bracketed]* parts with your real details, tester names, dates and screenshots.** The testing tables are ready for you to fill in as you run the program.

Title Page (rebuild this as page 1 in Word)

- **Project title:** Dojo Manager — Karate Student & Class Management System
- **Document:** Technical & Testing Document
- **Learner name:** *[your name]*
- **Exam number:** *[your exam number]*
- **Subject:** Information Technology (Grade 12 PAT 2026)
- **Date:** *[date]*

Table of Contents

---

1. Externally Sourced Code *(Rubric 4.1 — 3 marks)*

> **Be honest and specific here — your teacher will ask about this in the interview.** The rule is that **no more than 20% of the code may be borrowed or AI-generated**, and all of it must be listed. Below is a starting list; edit it to match exactly what you used.

#	Source / tool	What it was used for	Where in the code
1	Java Standard Library — `Desktop.browse()`	Opening Google Maps in the browser	`LocationPanel.openGoogleMaps()`
2	Java Standard Library — object serialisation (`ObjectOutputStream` / `ObjectInputStream`)	Saving and loading data to `.dat` files	`DataManager`
3	Java `java.time.LocalDate`	Getting today's weekday for "Today's classes"	`DataManager.getTodayName()`

#	Source / tool	What it was used for	Where in the code
4	AI assistant (Cursor)	Helped add Javadoc comments; created the `Validation` helper class; added some `DataManager` methods (instructor count, today's classes, the two text-report exports, and the class-clash check); suggested small UI improvements (sortable tables, double-click-to-edit); and helped draft these PAT documents	`Validation.java` ; the comments and the `hasClassClash` / export / helper methods in `DataManager.java` ; and the three documents

AI acknowledgement (complete this honestly with your teacher):

- **Tool used:** Cursor (an AI coding assistant).
- **What it helped with:** the items listed in row 4 above.
- **Example prompts I used:** *[write the actual instructions you gave, e.g. "Add Javadoc comments and simple input validation to my Java dojo program without changing how it works", and "help me write my PAT Design Document from my code"].*
- **My own work:** the original Dojo Manager application (the model classes, the `DataManager` storage, and the GUI screens) is my own work; the AI mainly added comments, the small `Validation` class, helper methods and documentation on top of it.
- **Declaration:** I understand all of the code and can explain it, and the borrowed + AI-generated **code** is 20% or less of the program, as required by the IEB. *[You must be able to defend this in the interview — go through the README's interview section first.]*

---

2. Critical Algorithms *(Rubric 4.2 — 3 marks)*

> Pseudocode only (no Java code allowed here). These two algorithms are the most important to how the program works.

Algorithm A — Saving and loading data (secondary storage)

Why it is critical: Without this, nothing the user enters would be remembered after closing the program. It is the heart of the "permanent storage" part of the PAT.

```
ON PROGRAM START:
    make sure the "dojo_data" folder exists
    FOR each data file (students, classes, dojo info):
        IF the file exists THEN
            TRY
                read the saved list/object from the file
            CATCH any error
                start with an empty list instead (do not crash)
    IF there are no students AND no classes THEN
        add sample data

ON ANY CHANGE (add / edit / delete):
    update the list held in memory
    write the whole list back to its file
```

Algorithm B — Checking for a class clash

Why it is critical: It stops the user from booking the same instructor in two places at once, which keeps the timetable sensible and is an example of defensive logic.

```
FUNCTION hasClassClash(id, day, time, instructor):
    FOR each existing class in the class list:
        IF the class is NOT the one being edited (different id)
           AND its day equals day
           AND its time equals time
           AND its instructor equals instructor THEN
            RETURN true      // a clash was found
    RETURN false             // no clash
```

(Optional third algorithm you can include — searching/filtering the table:)

```
FUNCTION refreshTable(searchText):
    clear the table
    FOR each record:
        IF searchText is empty
           OR the record's name/day/instructor contains searchText THEN
            add the record as a row in the table
```

---

3. Functional Testing *(Rubric 4.3 — 3 marks)*

> Do **at least TWO full sets** of testing. Fill in the tester, date and result. It is fine if something fails — just record it honestly and show progress.

Functional Test Set 1

- **Tester name:** *[name]* **Date:** *[date]*

#	Function tested	Steps	Expected result	Actual result	Pass/Fail
1	Add student	Add "Sam Jones", age 13, White Belt	Student appears in table and is saved	*[fill in]*	*[ ]*
2	Edit student	Change Sam's belt to Yellow	Table updates and saves	*[fill in]*	*[ ]*
3	Delete student	Delete Sam, confirm Yes	Student removed from table	*[fill in]*	*[ ]*
4	Search student	Type "green" in Find	Only green-belt students show	*[fill in]*	*[ ]*
5	Add class	Add "Kids", Monday 16:00, Sensei Lee	Class appears and saves	*[fill in]*	*[ ]*
6	Data persists	Close and reopen the program	All data is still there	*[fill in]*	*[ ]*

Functional Test Set 2

- **Tester name:** *[a different person, e.g. your cousin]* **Date:** *[date]*

#	Function tested	Steps	Expected result	Actual result	Pass/Fail
1	Class clash	Add a second class for Sensei Lee, Monday 16:00	Warning shown, class not added	*[fill in]*	*[ ]*
2	Sort table	Click the "Age" column header	Students sort by age	*[fill in]*	*[ ]*
3	Export list	Click "Export List"	`student_list.txt` is created	*[fill in]*	*[ ]*
4	Export timetable	Click "Export Timetable"	`class_timetable.txt` grouped by day	*[fill in]*	*[ ]*
5	Today's classes	Check Home tab	Correct classes for today are listed	*[fill in]*	*[ ]*
6	Open in Maps	Location tab → Open in Google Maps	Browser opens at the address	*[fill in]*	*[ ]*

---

4. Test Plan and Results for Two Input Variables *(Rubric 4.4 — 3 marks)*

The **two input variables** chosen (from the Defensive Programming in the code) are the **student Age** and the **student Phone number**. Each is tested with **standard, extreme and abnormal** data. Add a **before** and **after** screenshot for each test.

Input Variable 1 — Age (whole number, must be 4–100)

Type of data	Input	Expected result	Actual result	Screenshots
Standard	14	Accepted, student saved	*[fill in]*	[ADD before/after]
Extreme	4 (lowest allowed)	Accepted	*[fill in]*	[ADD before/after]
Extreme	100 (highest allowed)	Accepted	*[fill in]*	[ADD before/after]
Abnormal	2	Rejected: "realistic age (4–100)"	*[fill in]*	[ADD before/after]
Abnormal	"abc" (letters)	Rejected: "Age must be a whole number"	*[fill in]*	[ADD before/after]

Input Variable 2 — Phone (10–13 digits)

Type of data	Input	Expected result	Actual result	Screenshots
Standard	0821112233	Accepted	*[fill in]*	[ADD before/after]
Extreme	0821112233445 (13 digits)	Accepted	*[fill in]*	[ADD before/after]
Abnormal	12345 (too short)	Rejected: "valid phone number"	*[fill in]*	[ADD before/after]
Abnormal	08a11b2233 (letters)	Rejected: "valid phone number"	*[fill in]*	[ADD before/after]

---

5. Evaluation of the Program *(Rubric 4.5 — 3 marks)*

How well does the program meet its functions (from the Specifications)?

Overall, the program meets the functions listed in the Specifications Document. During development the application built and ran without errors, and each main function worked as intended:

- Students and classes can be added, edited, deleted, searched and sorted.
- All data is saved to file and reloads correctly when the program is reopened, which satisfies the core "permanent storage" requirement of the PAT.
- Input is validated (name, age, phone and email), and clear error messages are shown for invalid data.
- The dashboard shows live totals and the correct classes for the current day.
- The timetable warns about instructor clashes before saving.
- Both text reports (student list and weekly timetable) export successfully.

[After you run your own functional tests in sections 3 and 4, update this paragraph so it matches your actual results — and be honest about anything that did not work on your machine.]

Suggestions for any shortfalls

- If a test fails, note here what went wrong and how you would fix it (for example, a validation rule that is too strict, or a screen that needs a clearer label). If everything you tested worked, state: "All tested functions worked as expected."

Possible improvements / alternative solutions (for the future)

- Add an **attendance register** to tick off who attended each class.
- Add a simple **login screen** so only staff can open the program.
- Link students to the specific classes they attend.
- Store the data as **readable text/CSV or a database** instead of `.dat` files if the dojo later wants to open the data in Excel or share it.
- Add a **backup** button that copies the data files to another folder.

Overall conclusion

[Write 2–3 sentences: does the program solve the original problem for the dojo administrator, and are you satisfied it meets the PAT requirements?]